Zeluvi – Definição Arquitetural

Princípios e visão geral

A arquitetura proposta é um monólito modular stateless, exposto por uma API REST e consumido por um app web e um app mobile. Esse desenho entrega rápido, é simples de operar por uma equipe pequena e escala horizontalmente; se um módulo crescer demais, ele pode virar um serviço separado sem reescrever o resto.

1. **Stateless:** nenhuma sessão fica na memória do servidor. O estado vive no PostgreSQL, no Redis e no armazenamento de arquivos, então qualquer réplica atende qualquer requisição.
2. **API primeiro:** um único contrato OpenAPI serve web e mobile.
3. **Multi-tenant por condomínio:** todo dado carrega o identificador do condomínio, e o banco impede leitura entre condomínios.
4. **Módulos com fronteiras claras:** Identidade, Comunicação, Ocorrências, Serviços e IA/Automações.
5. **Trabalho pesado fora da requisição:** envio de notificações, transcrição e automações rodam em filas.
6. **Acessível por padrão:** telas seguem o guia de estilo (letra grande, contraste alto, alvo de toque de 56 px).
7. **Observável:** logs, métricas e rastreamento desde a primeira versão.

Backend stateless REST

O backend é uma aplicação Node.js com TypeScript (NestJS) que roda em várias réplicas idênticas atrás de um balanceador. Cada requisição traz tudo o que precisa (o token de acesso), então nenhuma réplica guarda sessão e qualquer uma pode ser criada ou removida a qualquer momento.

Camada	Responsabilidade
Controladores REST	Validam a entrada, checam a permissão do perfil e devolvem JSON
Serviços de domínio	Regras de cada módulo: Comunicação, Ocorrências, Serviços e IA
Repositórios	Acesso ao PostgreSQL, sempre filtrando pelo condomínio

Camada	Responsabilidade
Workers	Consomem as filas (notificações, transcrições, automações); mesma base de código, processo separado

Contratos de API

- REST sobre HTTPS com JSON e versão na URL (`/v1`). A documentação OpenAPI é gerada a partir do código.
- Recursos no plural: `/v1/ocorrencias` , `/v1/comunicados` , `/v1/reservas` , `/v1/encomendas` , `/v1/moradores` .
- Paginação por cursor e filtros por query string.
- Escritas que o celular pode repetir (como criar uma ocorrência) aceitam o cabeçalho `Idempotency-Key` , para não duplicar registros.
- Erros padronizados (RFC 9457), com mensagens em português simples.

Autenticação e autorização

- Login com e-mail e senha (hash Argon2), token de acesso JWT de 15 minutos e refresh token rotativo.
- O JWT carrega usuário, condomínio e perfil (morador, síndico ou portaria). Cada rota confere o perfil, conforme a matriz de acesso do mapa de telas.
- Limite de requisições por usuário e por IP, contado no Redis.

Frontend web e mobile

Web e mobile consomem a mesma API e compartilham código TypeScript, o que mantém uma única linguagem de ponta a ponta e deixa os dois apps consistentes entre si.

	Web	Mobile
Tecnologia	React com TypeScript (Vite)	React Native (Expo) com TypeScript
Foco	Painel do síndico, rotina da portaria e acesso do morador pelo navegador	Uso diário do morador, com notificações push
Navegação	Menu lateral de 280 px	Barra inferior com 5 botões, todos com ícone e palavra
Dados	React Query guarda em cache as respostas da API	Mesma biblioteca; leitura de avisos e reservas disponível mesmo com sinal fraco

O que é compartilhado: o cliente da API (gerado a partir do OpenAPI), os tipos e as validações de formulário, e os tokens de design do guia de estilo (cores, fontes, espaçamentos e raios).

Acessibilidade: a meta é WCAG 2.2 nível AA. Isso inclui texto de 18 px que pode crescer até 200% sem quebrar a tela, contraste de pelo menos 7 para 1, alvos de toque de 56 px, ícones sempre acompanhados de texto e confirmação antes de ações destrutivas. Cada release passa por verificação automática (axe) e por teste manual com leitor de tela (VoiceOver e TalkBack).

Tempo real: o mobile recebe avisos por notificação push e o web por WebSocket; esse canal é detalhado no Mapeamento de Integrações.

Stack tecnológica

A stack prioriza uma linguagem única de ponta a ponta, serviços gerenciados para a equipe não operar infraestrutura e tecnologias maduras com bastante mão de obra disponível. As escolhas são propostas e devem ser validadas pela equipe antes da defesa.

Camada	Escolha	Por que escolhemos e o que descartamos
Backend	Node.js com TypeScript (NestJS)	Mesma linguagem do front, módulos nativos e OpenAPI gerado do código. Descartamos Java/Spring, mais pesado para uma equipe pequena.
Web	React com TypeScript	Ecossistema grande e reuso de código com o mobile. Descartamos Angular por não compartilhar código com o app.
Mobile	React Native (Expo)	Um único código para Android e iOS. Descartamos o desenvolvimento nativo, que dobraria o trabalho.
Banco de dados	PostgreSQL	Transações ACID, segurança por linha, particionamento e réplicas de leitura. Descartamos MongoDB, pois os dados têm relações fortes (unidade, morador, reserva).
Cache e limites	Redis	Cache de leitura, contadores de limite de requisições e dados temporários.

Camada	Escolha	Por que escolhemos e o que descartamos
Filas	Amazon SQS com SNS	Gerenciado, sem broker para operar. Descartamos Kafka, complexo demais para esta carga.
Arquivos	Amazon S3 com CloudFront	Armazenamento barato e entrega por CDN, sem passar pela API.
IA	APIs de provedores de modelos de linguagem e de transcrição	Evita treinar e hospedar modelos próprios; as chamadas rodam nos workers.
Execução	AWS, com contêineres no ECS Fargate	Autoescalonamento sem administrar servidores. Descartamos Kubernetes pelo custo operacional.
Entrega contínua	GitHub Actions	O repositório do projeto já está no GitHub.

Banco de dados, cache e arquivos

O PostgreSQL é a única fonte da verdade: um primário recebe todas as escritas, um standby em outra zona assume se ele falhar, e réplicas de leitura atendem consultas pesadas. Redis e arquivos são apoio, e perder o Redis não perde dados.

PostgreSQL (Amazon RDS, Multi-AZ)

- Um banco com esquema único para todos os condomínios, com o identificador do condomínio em toda tabela e segurança por linha. É mais simples de operar do que um banco por condomínio e atende o volume previsto; se um condomínio crescer demais, as tabelas podem ser particionadas por ele.
- Pool de conexões (RDS Proxy), porque muitas réplicas stateless abrem muitas conexões ao mesmo tempo.
- Migrações versionadas, executadas na entrega e compatíveis com a versão anterior do código, para publicar sem parar o sistema.
- Backups automáticos com restauração para qualquer ponto no tempo, e teste de restauração periódico.

Redis (Amazon ElastiCache)

- Cache de leitura, contadores de limite de requisições e dados temporários com validade.

Arquivos (Amazon S3)

- Fotos de ocorrências e anexos sobem direto do app para o S3 por endereços pré-assinados de curta duração, sem passar pela API.
- Tipo e tamanho dos arquivos são validados, e a entrega ao usuário usa CDN.

Infraestrutura e ambientes

Os três ambientes usam a mesma definição de infraestrutura, o que evita o "na minha máquina funciona" e deixa a produção previsível.

Ambiente	Para que serve	Como é montado
Desenvolvimento	Cada pessoa da equipe programa e testa localmente	Docker Compose com PostgreSQL, Redis e fila simulada, em dados fictícios
Homologação	Testes automáticos, validação da equipe e ensaio de carga	Cópia reduzida da produção, na AWS, com dados fictícios
Produção	Uso real pelos condomínios	Duas zonas de disponibilidade, banco Multi-AZ, réplicas de leitura, autoescalonamento e alertas

- **Região:** AWS São Paulo, para manter os dados no Brasil.
- **Rede:** VPC com o balanceador em sub-rede pública e o restante em sub-redes privadas.
- **Infraestrutura como código:** Terraform, versionado junto com o projeto.
- **Entrega:** GitHub Actions com testes, build, homologação e publicação gradual em produção, com retorno automático em caso de falha.

Decisões arquiteturais e alternativas descartadas

Cada decisão abaixo tem um motivo e uma condição para ser revista, o que ajuda a defender a arquitetura com argumentos e não só com preferências.

Decisão	Motivo	Alternativa descartada	Quando revisar
Monólito modular	Uma só entrega, menos custo operacional e fácil de entender	Microsserviços desde o início	Quando um módulo (como a IA) precisar escalar ou ser entregue sozinho
Autenticação por token (stateless)	Qualquer réplica atende qualquer requisição	Sessão guardada no servidor	Se for exigido revogar o acesso de um usuário de forma instantânea
Um banco para todos os condomínios	Operação simples, com segurança por linha como trava	Um banco por condomínio	Se um cliente exigir isolamento físico dos dados
REST com JSON	Contrato claro, cache HTTP e ferramentas maduras	GraphQL	Se as telas passarem a precisar de muitas combinações de dados
Fila gerenciada (SQS)	Sem broker para operar	Kafka	Se for preciso reprocessar um grande histórico de eventos
React Native	Um código para os dois sistemas móveis	Apps nativos separados	Se o desempenho ou um recurso nativo exigir